

Actividad de Investigación y Práctica: Balanceo Compuesto en Árboles AVL y Exposición de Estructuras vía Web APIs

Duración estimada: 60 minutos

Modalidad: Individual

Entregable sugerido: Un documento en formato Markdown (.md) que contenga la resolución de la investigación teórica (con referencias cruzadas al material de soporte) y los fragmentos de código de la API funcional.

Parte 1: Investigación Teórica y Análisis de Casos

Con base en el material de la *Sesión 10: Rotaciones Dobles en Árboles AVL* y los conceptos de servicios web, responde de manera concisa a los siguientes apartados:

1. El Límite de las Rotaciones Simples y Desbalanceo en "Zig-Zag"

- **El Problema Cruzado:** Explica por qué las rotaciones simples (como RLL o RRD) fallan y solo cambian la inclinación de lado cuando se insertan secuencias cruzadas como 30, 10, 20. Define matemáticamente bajo qué condiciones de Factores de Equilibrio (FE) del nodo padre e hijo se gatilla una Rotación Doble Izquierda-Derecha (RID).
- **Principio DRY (Don't Repeat Yourself):** Explicar la ventaja de ingeniería de software que tiene implementar las operaciones compuestas (RID y RDI) reutilizando las primitivas de rotación simple en lugar de reasignar punteros manualmente desde cero.

2. Fundamentos de Arquitectura Web y Protocolo HTTP

- **El Modelo Cliente-Servidor:** Tomando en cuenta la transición hacia arquitecturas web, describe brevemente qué componentes interactúan cuando un cliente web solicita un recurso al servidor y de qué manera viajan los datos en una petición (*Request*) y una respuesta (*Response*).
- **Semántica de Operaciones:** Define la diferencia técnica entre los métodos HTTP GET y POST. Explica cuál de ellos está diseñado para la recuperación de la estructura de datos y cuál para la mutación o inserción de nuevos elementos.

Parte 2: Implementación Práctica: API de Simulación AVL

Para consolidar los algoritmos de la presentación, desarrollarás una Web API en C# utilizando **ASP.NET Core (Minimal APIs)**. La API expondrá en memoria una estructura autorreferenciada básica que simulará las reglas de balanceo compuesto de la sesión.

Instrucciones de Desarrollo:

1. Inicialización del Entorno:

Crea un nuevo proyecto de API web desde tu terminal:

Bash

```
dotnet new webapi -o ApiAvlSimulacion
```

```
cd ApiAvlSimulacion
```

2. **El Modelo del Nodo (Estructura de la Sesión):** Modela la estructura del nodo de acuerdo con el comportamiento de la presentación, incluyendo la propiedad del dato y el factor de equilibrio implícito:

```
public class NodoAVL
{
    public int Id { get; set; } // Actúa como el Dato/Llave
    public string Etiqueta { get; set; } = string.Empty;
    public int Altura { get; set; } = 1;
}
```

3. **Implementación de Endpoints (GET y POST) en Program.cs:** Configura el almacenamiento en memoria y escribe los endpoints de simulación que representen el escenario analizado en la slide 5 (valores 30, 10 y 20).

```
var builder = WebApplication.CreateBuilder(args);
```

```
var app = builder.Build();
```

```
// Simulación del estado del árbol en memoria
```

```
var estadoArbol = new List<NodoAVL>
```

```
{
    // Estado inicial desbalanceado en Zig-Zag (Slide 5)
    new NodoAVL { Id = 30, Etiqueta = "Nodo Raíz (Abuelo) - FE: -2" },
    new NodoAVL { Id = 10, Etiqueta = "Hijo Izquierdo - FE: +1" }
};
```

```
// 1. ENDPOINT GET: Recupera la estructura física del árbol actual
```

```
app.MapGet("/api/arbol", () => Results.Ok(estadoArbol));
```

```
// 2. ENDPOINT POST: Simula la inserción que gatilla el balanceo compuesto
```

```
app.MapPost("/api/arbol/insertar", (NodoAVL nuevoNodo) =>
```

```
{
    // Validación básica de la llave
    if (nuevoNodo.Id <= 0) return Results.BadRequest("ID de nodo inválido.");
```

```
    // Simulación de la lógica del motor de inserción (Slide 8 y 10)
```

```
    // Al insertar el 20, se detecta el caso cruzado Izquierda-Derecha
```

```
    if (nuevoNodo.Id == 20)
```

```

{
    estadoArbol.Clear();
    // El resultado de la rotación RID balancea perfectamente el árbol (Slide 9)
    estadoArbol.Add(new NodoAVL { Id = 20, Etiqueta = "Nueva Raíz Balanceada (RID) - FE: 0" });
    estadoArbol.Add(new NodoAVL { Id = 10, Etiqueta = "Hijo Izquierdo - FE: 0" });
    estadoArbol.Add(new NodoAVL { Id = 30, Etiqueta = "Hijo Derecho - FE: 0" });

    return Results.Created("/api/arbol", new { Mensaje = "Rotación RID ejecutada con éxito. Estabilidad total lograda.", Estructura = estadoArbol });
}

// Inserción tradicional sin rotación compuesta
estadoArbol.Add(nuevoNodo);
return Results.Created($"api/arbol/{nuevoNodo.Id}", nuevoNodo);
});

app.Run();

```

4. Pruebas de Verificación Obligatorias:

- **Paso A (GET):** Consulta <http://localhost:XXXX/api/arbol> para verificar la existencia del estado inicial desbalanceado en forma de Zig-Zag.
- **Paso B (POST):** Envía un POST al endpoint `/api/arbol/insertar` con el siguiente cuerpo JSON para disparar el balanceo analizado:

```

{
  "id": 20,
  "etiqueta": "Nieto Derecho"
}

```

- **Paso C (Verificación):** Confirma que la API responda con un estado 201 Created y devuelva el nuevo orden en memoria donde el nodo 20 ha pasado a ser la raíz del subárbol, neutralizando el desbalance.